

ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 13/04

- Repaso: Programación estructurada
- Repaso: Estructuras de Control: Secuencia.
- Selección múltiple: Switch.
- Errores frecuentes en programación.

2

AEDD – Asistencia del día 13/04/2026

8vb2rc

3

AEDD – Tareas semanales!!!!

Vincularse al curso en Omega Up: Comision F-2026-Anual.

Esto nos sirve para llevar la materia al día, y además para ir evaluando que contenidos son comprendidos y cuales deben ser revisados.

4

Metodología de Programación Estructurada

Metodología que consiste en escribir programas con las siguientes reglas:

- El programa tiene un diseño modular
- Los módulos son diseñados de modo descendente
- Cada módulo se codifica utilizando las tres estructuras de control básicas:
 - Secuencia
 - Selección
 - Repetición

5

Metodología de Programación Estructurada

Teorema fundamental de la PE (Böhm y Jacopini, 1966):

“Todo programa propio puede ser escrito usando solamente las estructuras de control básicas: secuencia, selección y repetición”

Propio:

- Tiene un único punto de entrada y un único punto de salida;
- Existen caminos desde la E a la S que se pueden seguir y que pasan por todas las partes del programa;
- Todas las instrucciones son ejecutables y no hay bucles infinitos.

6

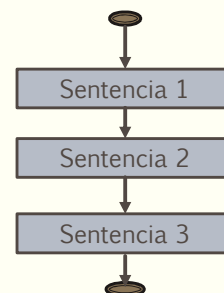
Estructuras de control

- Controlan el flujo de ejecución de las acciones de un programa
- Permiten combinar instrucciones o sentencias individuales en una simple unidad lógica con un único punto de entrada y un único punto de salida
- Se clasifican en tres tipos:
 - Secuencia (Composición secuencial)
 - Selección (Composición alternativa), y
 - Repetición (Composición iterativa).

7

Estructuras de control : SECUENCIA

- Conjunto de sentencias que se ejecutan una luego de la otra (flujo secuencial).



- En C++: conjunto de sentencias que finalizan con ';' y están encerradas entre llaves.

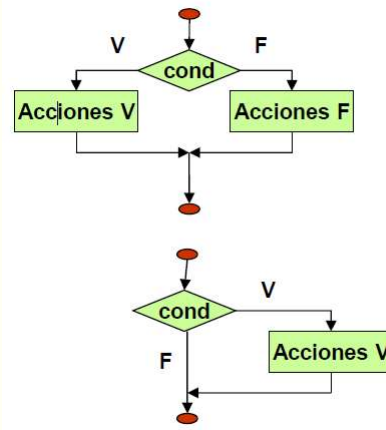
```

{
  int x, z, y;
  x = 1 + b;
  z = y * 28;
}
  
```

8

Estructura de control: Selección

- **Selección completa:** Se evalúa una expresión booleana y según sea verdadera o falsa, se ejecuta una acción u otra; por cualquier caso la ejecución continúa con la sentencia siguiente al if.



- **Selección incompleta:** Sólo se desea realizar una acción en caso de que la expresión booleana sea verdadera y no hay acción por el falso.

Las acciones pueden ser una sentencia que termina en “;” o un grupo de sentencias encerradas entre llaves.

9

Estructura de control: Selección

- En C++:

SELECCIÓN
COMPLETA

```
if (expresión)
    acción_verdadero;
else
    acción_falso;
```

SELECCIÓN
INCOMPLETA

```
if (expresión)
    acción_verdadero;
```

10

Selección anidada

- Permite evaluar varios casos. Una sentencia if es anidada cuando la acción de la rama verdadera y/o la rama falsa, son también una sentencia if.

Importante: Las llaves se deben agregar para indicar al compilador C++ la asociación de los if-else.

El compilador C++ siempre asocia un else con el if inmediatamente anterior (el que le precede). Si queremos que se realice otra asociación diferente, deberemos colocar llaves.

- En C++:

```
if (exp1)
    acción 1;
else
    if (exp2)
        acción 2;
    else
        acción 3;
```

Se abre la rama falsa

```
if (exp1)
    if (exp2)
        acción 1;
    else
        acción 2;
else
    acción 3;
```

Se abre la rama verdadera

- El if exterior o los if interiores pueden ser incompletos. Ejemplos:

```
if (exp1) {
    if (exp2)
        acción 2;
    else
        acción 3;
}
```

El if exterior es incompleto

```
if (exp1) {
    if (exp2)
        acción 1;
}
else
    acción 3;
```

El if interior es incompleto

11

Selección anidada: Ejemplo

- Se ingresa un número entero, y queremos incrementar un contador general, y un contador de números positivos, o negativos, o nulos según corresponda

```
{
    cin >> n;
    if (n > 0)
        ContPos ++;
    else
        if (n < 0)
            ContNeg ++;
        else
            ContNul ++;
    ContGral ++;
}
```

Que diferencia hay con esta otra forma?

```
cin >> n;
if (n > 0)
    ContPos ++;
if (n < 0)
    ContNeg ++;
if (n == 0)
    ContNul ++;
ContGral ++;
```

12

Selección Múltiple: Sentencia Switch

- Se emplea para los casos en que se debe seleccionar una de entre múltiples alternativas, en forma exclusiva, basada en el valor de una variable o expresión denominada selector o expresión de control, la cual sólo puede ser de tipo int o char.
- El mecanismo se completa con la sentencia “break” que hace que siga la ejecución en la siguiente sentencia, dentro del programa, en este caso la sentencia siguiente al switch.
- El switch es muy útil para transformaciones de datos para salida y para el manejo de menú en las interfases de los programas.

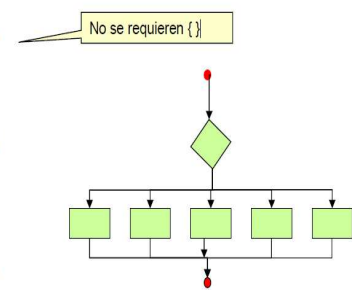
13

Selección Múltiple: Sentencia Switch

- Cada etiqueta es un valor único, constante (int o char) y debe tener un valor diferente de los otros.
- Se van evaluando las etiquetas y cuando alguna coincide con el valor del selector se ejecutan todas las sentencias asociadas.
- Si el valor del selector no es igual a ninguna etiqueta, no se ejecuta ninguna opción, a menos que se use el default.

En C++:

```
switch (selector) {
    case etiqueta_1:
        sentencias;
        break;
    case etiqueta_2:
        sentencias;
        break;
    .....
    default: /* opcional */
        sentencias;
        break;
}
```



14

Selección Múltiple: Sentencia Switch : Ejemplo

- Ejemplo 3: Muestre según el año ingresado (entre 1 a 5), cuantos años les faltan cursar a un alumno de la carrera de ingeniería.

```
int main() {
    int anio_cursado = 1;
    cout << "Ingrese el año actual que esta cursando: " << endl;
    cin >> anio_cursado;
    switch (anio_cursado) {
        case 1: cout << "Te faltan cursar 4 años";
                break;
        case 2: cout << "Te faltan cursar 3 años";
                break;
        case 3: cout << "Te faltan cursar 2 años";
                break;
        case 4: cout << "Te falta cursar 1 año";
                break;
        case 5: cout << "Estas cursando el ultimo año";
                break;
        default: cout << "Año de cursado incorrecto";
    }
    return 0;
}
```

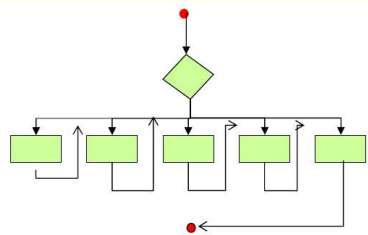
15

Selección Múltiple: Sentencia Switch sin BREAK

Tiene un comportamiento en cascada.

En C++:

```
switch (selector) {
    case etiqueta_1:
        sentencias;
    case etiqueta_2:
        sentencias;
    .....
    default: /* opcional */
        sentencias;
}
```



Se van evaluando las etiquetas y cuando el valor del selector es igual a una de las etiquetas, se ejecutan todas las sentencias a partir de la que corresponde a ese selector.

16

Selección Múltiple: Sentencia Switch sin BREAK

Tarea: Modifique el ejemplo 3, sacando los breaks.... ¿Cómo se comporta?
¿Qué muestra como salidas cuando ingresamos 2?

```
Ingrese el año actual que esta cursando:  
2  
Te faltan cursar 3 años  
Te faltan cursar 2 años  
Te falta cursar 1 año  
Estas cursando el ultimo año  
Año de cursado incorrecto
```

17

Errores frecuentes

- Usar = en lugar de == en las condiciones.
- Confundir la asociación de else con el if correspondiente :
Solución: encerrar entre llaves.
- Que no haya concordancia entre el tipo del selector y el tipo de las etiquetas, en el switch.
- Omitir el break cuando sea requerido.

18

AEDD – Tareas semanales!!!!

Vincularse al curso en Omega Up: Comision F-2026-Anual.

19

AEDD – Tareas semanales!!!!

Vincularse al curso en Omega Up: Comision F-2026-Anual.

Una pequeña ayuda para la tarea de esta semana: Estructuras repetitivas

```
int main(){
    int numero = 1, suma = 0;
    while (numero <= 10){
        suma += numero;
        numero++;
    }

    cout << "La suma de los 10 primeros numeros naturales da: " << suma;
    return 0;
}
```

20

AEDD – Asistencia del día 13/04/2026

8vb2rc

21

AEDD – Otras tareas !!!!!

Capítulo 4

Libro: “Fundamentos de Programación con el
Lenguaje de Programación C++” – Benjumea-
Roldan

Capítulo 4

Libro: “C++ para ingeniería y ciencias”, 2da
Edición. Gary J. Bronson

Capítulo 4

Libro: “Cómo programar en C++” – Harvey M.
Deitel y Paul J. Deitel

22

Algoritmos y Estructuras de Datos – AEDD – Agenda 13/04

- ✓ Repaso: Programación estructurada
- ✓ Repaso: Estructuras de Control: Secuencia.
- ✓ Selección múltiple: Switch.
- ✓ Errores frecuentes en programación.

¡Objetivo de la primer parte del día alcanzado!

23

Algoritmos y Estructuras de Datos – AEDD – Agenda 13/04

- Tipo String.
- Entrada/Salida del Tipo String.
- Comparación/Concatenación/Longitud.
- Acceso a los caracteres de una cadena.
- Operaciones con cadenas tipo string.
- Eliminar/Insertar.
- Biblioteca ctype

24

Cadena de Caracteres en C++: Tipo STRING

El lenguaje C++ dispone de la biblioteca estándar <string>, que proporciona el tipo string para representar cadenas de caracteres.

El tipo string dispone de operadores predefinidos que permiten manejar cadenas de caracteres de forma muy simple e intuitiva.

- Sin longitud máxima (gestión automática de la memoria)

Muchas funciones de utilidad (biblioteca string)

El tipo string puede ser utilizado para definir constantes simbólicas, variables o parámetros formales en los subprogramas.

En AEDD no emplearemos datos de tipo string como elementos (campos) de una estructura (struct) en Archivos.

Los strings de C++ hacen crecer el espacio de almacenamiento para acomodarse a los cambios de tamaño de los datos de la cadena por encima de los límites de la memoria asignada inicialmente.

25

Cadena de Caracteres en C++: Tipo STRING

Declaración e inicialización:

```
#include <iostream>
#include <string>
using namespace std;
const string AUTOR = "Jose Luis";

int main() {
    string nombre = "Pepe";
    cout << "Nombre: " << nombre << endl;
    nombre = AUTOR;
    cout << "Nombre: " << nombre << endl;
    return 0;
}
```

AUTOR:	J	o	s	e		L	u	i	s
	0	1	2	3	4	5	6	7	8

nombre:	P	e	p	e
	0	1	2	3

nombre:	J	o	s	e		L	u	i	s
	0	1	2	3	4	5	6	7	8

Si la definición de una variable de tipo string no incluye la asignación de un valor inicial, dicha variable tendrá como valor por defecto la cadena vacía ("").

26

Entrada/Salida del Tipo String

La entrada/salida de datos de tipo string se basa en el uso de los operadores >> y << sobre los flujos cin y cout.

El operador << sobre un flujo de salida cout muestra todos los caracteres que forman parte de la cadena.

El operador de entrada (>>) se comporta de la siguiente forma:

- Elimina los espacios en blanco que hubiera al principio de la entrada de datos
- Y lee dicha entrada hasta que encuentre algún carácter de espacio en blanco, que no será leído y permanecerá en el buffer de entrada hasta la próxima operación de entrada.

Como consecuencia de lo anterior, no es posible utilizar el operador >> para leer una cadena de caracteres que incluya algún carácter en blanco.

27

Entrada/Salida del Tipo String

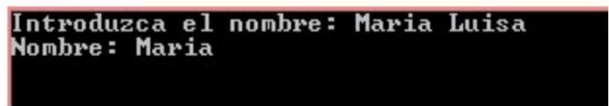
No es posible utilizar el operador >> para leer una cadena de caracteres que incluya algún carácter en blanco.

El carácter en blanco actúa como delimitador y fuerza el fin de la lectura.

Ejemplo:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombre;
    cout << "Introduzca el nombre: ";
    cin >> nombre;
    cout << "Nombre: " << nombre << endl;
    return 0;
}
```



```
Introduzca el nombre: Maria Luisa
Nombre: Maria
```

28

Tipo String: Entrada de cadena

Si se desea leer una secuencia de caracteres que incluya espacios en blanco, utilizaremos la función `getline` en lugar del operador `>>`.

- `getline()` lee y almacena en una variable de tipo `string` todos los caracteres del buffer de entrada, hasta leer el carácter de fin de línea (ENTER). No elimina espacios iniciales.

```
#include <iostream>
#include <string>
using namespace std;

int main(){
    string nombre;
    cout << "Introduzca el nombre: ";
    getline(cin, nombre);
    cout << "Nombre: " << nombre << endl;
    return 0;
}
```

```
Introduzca el nombre: Juan Antonio
Nombre: Juan Antonio
```

29

Tipo String: Entrada de cadena

Además, la función `getline` permite especificar el delimitador que marca el final de la secuencia de caracteres a leer.

```
const char DELIMITADOR = ',';

int main(){
    string nombre;
    cout << "Introduzca el nombre: ";
    getline(cin, nombre, DELIMITADOR);
    cout << "Nombre: " << nombre << endl;
    return 0;
}
```

```
Introduzca el nombre: Juan Anto,nio
Nombre: Juan Anto
```

30

Tipo String: Entrada de cadena

Además, la función `getline` permite especificar el delimitador que marca el final de la secuencia de caracteres a leer.

```
const char DELIMITADOR = ',';

int main(){
    string nombre;
    cout << "Introduzca el nombre: ";
    getline(cin, nombre, DELIMITADOR);
    cout << "Nombre: " << nombre << endl;
    return 0;
}
```

```
Introduzca el nombre: Juan Anto.nio
Nombre: Juan Anto
```

31

Tipo String: Problemas con getline

En ocasiones, cuando se utiliza una lectura con `getline`, después de una lectura previa con `>>`, podemos encontrarnos con un comportamiento que, aunque correcto, puede no corresponder al esperado intuitivamente.

```
#include <iostream>
#include <string>
using namespace std;

int main(){
    string nombre;
    int edad;
    for(int i = 0; i<5; i++){
        cout << "Introduzca el nombre: ";
        getline(cin, nombre);
        cout << "Introduzca la edad: ";
        cin >> edad;
        cout << "--> Edad: " << edad << " Nombre: " << nombre << endl;
    }
    return 0;
}
```

```
Introduzca el nombre: Juan Antonio
Introduzca la edad: 56
--> Edad: 56 Nombre: Juan Antonio
Introduzca el nombre: Introduzca la edad:
```

32

Solución: limpiar el buffer -ws

- El resultado es que, al leer el nombre, se lee una cadena vacía, sin que se detenga el programa para que el usuario introduzca el nombre de la persona.
- La solución a este problema es eliminar los caracteres de espacios en blanco (y fin de línea) del buffer de entrada. De esta forma el buffer estará realmente vacío y conseguiremos que la ejecución de getline() haga que el programa se detenga hasta que el usuario introduzca el nombre.

Hay diferentes formas de conseguir que el buffer se quede vacío.

- Utilizaremos el manipulador ws en el flujo cin, que extrae todos los espacios en blanco hasta encontrar algún carácter distinto (en ese momento la extracción se detiene).

33

Solución: limpiar el buffer -ws

```
int main(){
    string nombre;
    int edad;
    for(int i = 0; i<5; i++){
        cout << "Introduzca el nombre: ";
        cin >> ws; //Elimina los espacios en blanco y el fin de linea
        getline(cin, nombre);
        cout << "Introduzca la edad: ";
        cin >> edad;
        cout << "--> Edad: " << edad << " Nombre: " << nombre << endl;
    }
    return 0;
}
```

34

Solución: limpiar el buffer - ignore

Para eliminar los caracteres que pudiera contener el buffer (no únicamente espacios en blanco) después de la última operación de lectura de datos, usamos la función `cin.ignore()`.

- **`cin.ignore()`** elimina todos los caracteres del buffer de entrada en el flujo especificado, hasta que se haya eliminado el número de caracteres indicado en el primer argumento o bien se haya eliminado el carácter indicado en el segundo.

La sentencia `cin >> ws` se asocia a la función `getline` que le sigue, mientras que la sentencia `cin.ignore` se asocia a la sentencia de entrada `>>` que aparece antes.

35

Solución: limpiar el buffer - ignore

```
int main(){
    string nombre;
    int edad;
    for(int i = 0; i<5; i++){
        cout << "Introduzca el nombre: ";
        getline(cin, nombre);
        cout << "Introduzca la edad: ";
        cin >> edad;
        cin.ignore(10000, '\n'); //elimina todos los caracteres del buffer
        cout << "--> Edad: " << edad << " Nombre: " << nombre << endl;
    }
    return 0;
}
```

36

Comparación/Concatenación/Longitud

- Comparaciones lexicográficas con operadores relacionales

(==, !=, >, <, >=, <=):

```
if (nombre >= AUTOR) { /*...*/ }
```

- +: Concatenación de cadenas:

```
const string AUTOR = "José Luis" ;
int main ()
{
    string nombre = AUTOR + "López" ;
    nombre += "Vázquez" ;
    nombre += 'z' ;
    nombre = AUTOR + 's' ;
}
```

size() o length(): obtención de la longitud de la cadena (número de

caracteres): `unsigned ncar = nombre.size();`

```
if (nombre.size() == 0) { /*...*/ }
```

37

Comparación/Concatenación/Longitud

- Comparaciones lexicográficas con operadores relacionales

(==, !=, >, <, >=, <=):

```
if (nombre >= AUTOR) { /*...*/ }
```

- +: Concatenación de cadenas:

```
const string AUTOR = "José Luis" ;
int main ()
{
    string nombre = AUTOR + "López" ;
    nombre += "Vázquez" ;
    nombre += 'z' ;
    nombre = AUTOR + 's' ;
}
```

size() o length(): obtención de la longitud de la cadena (número de

caracteres): `unsigned ncar = nombre.size();`

```
if (nombre.size() == 0) { /*...*/ }
```

38

Acceso a los caracteres de una cadena

Mediante la función `at` (índice): Devuelve el carácter en la posición especificada y lanzará una excepción si se accede a una posición inexistente.

```
int main() {
    string cadena1="Algoritmos";
    cout << cadena1.at(2) << endl;
    cout << cadena1.at(17) << endl;
    return 0;
}
```

Aunque lanza una excepción si hay algo mal.

39

Operaciones con cadenas tipo String

- **`substr(posición, longitud)`:** obtiene una nueva subcadena a partir de posición. Si no se especifica una longitud, o si esta excede al número de caracteres que hay desde posición, entonces se devuelve la subcadena desde `i` hasta el final.

```
string cad = "abcdefg";
cout << cad.substr(2, 3); // Muestra cde
```

- **`swap (cadena2)`:** intercambia `cadena2` con otra cadena.

`cadena1.swap(cadena2)`

```
*** Antes del cambio ***
Cadena1: Hola
Cadena2: Adios

*** Despues del cambio ***
Cadena1: Adios
Cadena2: Hola
```

40

Operaciones con cadenas tipo String

- **find (subcadena):** devuelve la posición de la 1era ocurrencia de subcadena en la cadena

```
string cad = "Olala";
cout << cad.find("la"); // Muestra 1
```

- **rfind (subcadena):** devuelve la posición de la última ocurrencia de subcadena en la cadena

```
string cad = "Olala";
cout << cad.rfind("la"); // Muestra 3
```

41

Eliminación e Inserción

- **erase (posición, cantidad):** elimina cantidad caracteres, a partir de posición. Si no se especifica una cantidad, entonces se elimina hasta el final de la cadena.

```
string cad = "abcdefgh";
cad.erase(3, 4); // cad ahora contiene "abch"
```

- **insert (posición, cadena2):** inserta cadena2, a partir de posición:

```
string cad = "abcdefgh";
cad.insert(3, "123"); // cad ahora contiene "abc123defgh"
```

42

Biblioteca para manejar caracteres

- La biblioteca <cctype> proporciona principalmente operaciones sobre los valores de tipo char:

Prototipo de la función	Descripción	Ejemplo
int isalpha(charExp)	Devuelve verdadero (número entero diferente de cero) si charExp evalúa una letra; de lo contrario, devuelve falso (número entero cero)	isalpha('a')
int isalnum(charExp)	Devuelve verdadero (número entero diferente de cero) si charExp evalúa una letra o un dígito; de lo contrario, devuelve falso (número entero cero)	char key; cin >> key; isalnum(key);
int isupper(charExp)	Devuelve verdadero (número entero diferente de cero) si charExp evalúa una letra mayúscula; de lo contrario, devuelve falso (número entero cero)	isupper('a')
int islower(charExp)	Devuelve verdadero (número entero diferente de cero) si charExp evalúa una letra minúscula; de lo contrario, devuelve falso (número entero cero)	islower('a')
int isdigit(charExp)	Devuelve verdadero (número entero diferente de cero) si charExp evalúa un dígito (0 a 9); de lo contrario, devuelve falso (número entero cero)	isdigit('a')

43

Biblioteca para manejar caracteres

- La biblioteca <cctype> proporciona principalmente operaciones sobre los valores de tipo char:

Prototipo de la función	Descripción	Ejemplo
int isalpha(charExp)	Devuelve verdadero (número entero diferente de cero) si charExp evalúa una letra; de lo contrario, devuelve falso (número entero cero)	isalpha('a')
int isalnum(charExp)	Devuelve verdadero (número entero diferente de cero) si charExp evalúa una letra o un dígito; de lo contrario, devuelve falso (número entero cero)	char key; cin >> key; isalnum(key);
int isspace(charExp)	Devuelve verdadero (número entero diferente de cero) si charExp evalúa un espacio; de lo contrario, devuelve falso (número entero cero)	isspace(' ')
int toupper(charExp)	Devuelve el equivalente en mayúscula si charExp evalúa un carácter en minúscula; de lo contrario, devuelve el código de carácter sin modificación	toupper('a')
int tolower(charExp)	Devuelve el equivalente en minúscula si charExp evalúa un carácter en mayúscula; de lo contrario, devuelve el código de carácter sin modificación	tolower('A')

44

Algoritmos y Estructuras de Datos – AEDD – Agenda 13/04

- ✓ Tipo String.
- ✓ Entrada/Salida del Tipo String.
- ✓ Comparación/Concatenación/Longitud.
- ✓ Acceso a los caracteres de una cadena.
- ✓ Operaciones con cadenas tipo string.
- ✓ Eliminar/Insertar.
- ✓ Biblioteca ctype

¡Objetivo de la segunda parte del día alcanzado!

45

AEDD – Otras tareas !!!!!

Capítulo 6 (El tipo string)

Libro: Benjumea-Roldan

Universidad de Málaga

Capítulo 7 (7.2 La clase string)

Libro: “C++ para ingeniería y ciencias” 2da edición -

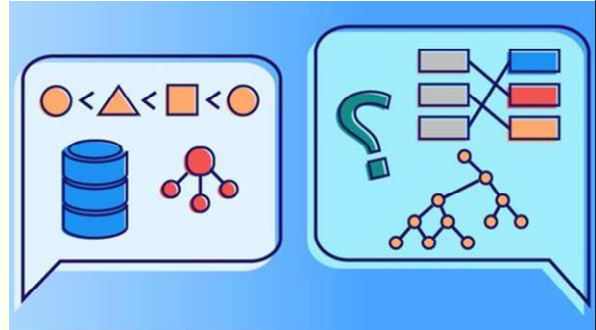
Gary J. Bronson

46

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¿DUDAS?



47

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2025 – Cursado Anual

¡Buen fin de semana!



48